

cMCP: Verifiable Policy Enforcement for AI Agent Tool Calls

Rishabh Poddar Aaron Fulkerson Imran Siddique
OPAQUE Systems

<https://github.com/agenttrust-io/cmcp>

September 27, 2026

Technical report, version 1

Abstract

cMCP (Confidential MCP) proposes a gateway for policy evaluation at the Model Context Protocol tool-call boundary. The gateway evaluates Cedar policies before dispatch and emits signed records binding policy measurements, session identifiers, and tool-call evidence. Its design combines policy hashing, monotonic session sensitivity, tool-catalog pinning, and audit-chain checks. This technical report preserves software-only experiments for those mechanisms, including synthetic healthcare call traces and simulated attestation paths. The measurements illustrate behavior under the stated inputs and assumptions; they do not validate a hardware TEE, establish complete information-flow tracking, or show that every tool invocation was captured. Hardware provenance requires independent attestation appraisal and signing-key binding. Replay and omission detection additionally depend on freshness checks and an expected session or log boundary. The report distinguishes these deployment requirements from the software behavior measured here.

Scope of this edition. This is an author-prepared technical report, not peer reviewed and not a normative specification. It revises a manuscript originally dated June 23, 2026; the recorded experiments include later runs and have not been rerun for this edition. Architectural requirements describe intended checks, not proof that the historical SDK implements every check. Consult the linked project specification and limitations before implementing the design.

Patent Pending.

Contents

1	Introduction	3
2	Background	4
2.1	Model Context Protocol	4
2.2	Trusted Execution Environments	4
2.3	TRACE Standard	4
2.4	Cedar Policy Language	5
2.5	Threat Model	5
3	cMCP Design	5
3.1	Architecture Overview	5
3.2	Tool Call Flow	6
3.3	Session Sensitivity State Machine	7
3.4	Tool Catalog Pinning	7
3.5	TRACE Claim Generation	7
3.6	Cross-Organizational Attestation (Phase 2)	8

4	Security Properties	8
5	Evaluation	10
5.1	Property 1: TEE-Measured Policy Enforcement	11
5.2	Property 2: Session Sensitivity Tracking	11
5.3	Property 3: Tool Catalog Rug-Pull Detection	12
5.4	Property 4: TRACE Claim Key Binding	12
5.5	Property 5: Temporal Adjacency Provenance	12
5.6	Property 6: Cross-Organizational Attestation Chain	13
5.7	Performance	13
5.8	Summary and CI Coverage	14
5.9	Limitations	14
6	Related Work	15
7	Conclusion	16

1 Introduction

AI agents that use tools are rapidly becoming the dominant pattern for deploying language models in production. A single user request may cause an agent to invoke a dozen tools: reading email, querying a database, calling an external API, posting to a messaging system. Each tool call crosses a trust boundary. The tool may be provided by a third party, run on a different server, or handle regulated data. Yet the agent’s orchestration layer has no standard mechanism for verifying what policies governed the call, who authorized the invocation, or whether the returned data places compliance obligations on everything the agent does next.

Model Context Protocol (MCP) [Anthropic, 2024] formalizes this interaction as a JSON-RPC protocol with standardized tool discovery and invocation semantics. MCP has seen rapid adoption: within a year of publication, hundreds of MCP servers expose tools ranging from filesystem access to medical record retrieval. This adoption makes the governance gap concrete. When an agent calls `ehr.get_patient` through an MCP server and the response contains protected health information, nothing in the MCP protocol tells the agent, its orchestrator, or a downstream auditor: which policy authorized this call; whether the response sensitivity has been tracked; whether the tool definition matches what was reviewed at deployment time.

cMCP addresses this gap. It is a TEE-based gateway that sits between an MCP client and one or more MCP servers. Every tool call passes through cMCP, which evaluates Cedar [Cutler et al., 2024] policies before dispatch and, after the call completes, appends a signed entry to a hash-chained audit log. At session close, cMCP produces a TRACE Claim [Poddar et al., 2026]: a cryptographically signed attestation that binds the session’s call record to the TEE instance that processed it and to the policy bundle in effect at startup.

The central thesis of this paper is that hardware attestation, applied at the MCP boundary, can provide verifiable governance properties that cannot be achieved through software-only interception. We validate six such properties through reproducible experiments, each backed by unit tests that run in CI without hardware TEE; the experiments establish the software-checkable behavior of each property, while the binding to physical hardware is architectural and is stated as a limitation (Section 5).

Contributions

- cMCP architecture. A TEE gateway design that is transparent to existing MCP clients and servers, requiring no protocol changes.
- TEE-measured policy enforcement (Property 1). Policy bundles are measured into the TEE at startup. Any runtime modification raises `PolicyHashMismatch` and invalidates the TRACE Claim signature.
- Session sensitivity tracking (Property 2). A monotonically increasing session sensitivity state machine detects PHI cross-boundary violations that per-call policy evaluation cannot see, at a quantified false-discovery cost.
- Rug-pull detection (Property 3). Tool catalog entries are hash-pinned at TEE startup. Any post-approval modification to a tool’s description or input schema triggers fail-closed rejection before the call is dispatched.
- TRACE Claim key binding (Property 4). Each TRACE Claim carries a nonce formed from the RFC 7638 JWK Thumbprint of the TEE public key and a random salt, committed into the hardware report. A verifier re-derives the thumbprint from the claim’s confirmation key to bind the report to the signing key; the session is bound through the signed claim body. Together these prevent replay, forgery, and selective disclosure of claims across sessions or instances.
- Temporal adjacency provenance (Property 5). cMCP records a call graph where edges represent temporal adjacency. This is a conservative provenance approximation: zero false negatives by construction, with a quantified false positive rate.
- Cross-organizational attestation chains (Property 6). A dual-TEE protocol links independent Phase 1 (gateway) and Phase 2 (server) TRACE Claims through a shared session identifier and independently

verifiable nonces.

Paper structure

Section 2 covers MCP mechanics, TEE attestation, the TRACE standard, Cedar policies, and the threat model. Section 3 describes the cMCP architecture and its key mechanisms. Section 4 formally states the six security properties. Section 5 presents experimental validation of each property with quantitative results. Section 6 surveys related work. Section 7 concludes.

2 Background

2.1 Model Context Protocol

Model Context Protocol (MCP) [Anthropic, 2024] is a JSON-RPC 2.0 protocol that standardizes how AI agents discover and invoke tools hosted on external servers. A tool is a named function with a JSON Schema describing its parameters. MCP defines three primitive operations: `tools/list` to enumerate available tools, `tools/call` to invoke one, and `resources/read` to retrieve structured data.

An MCP session begins when a client establishes a transport connection (HTTP with Server-Sent Events, or stdio) to an MCP server and exchanges `initialize` messages. The server advertises its tool catalog; the client may cache it. Subsequent tool calls are stateless from the protocol’s perspective: each `tools/call` carries the tool name and a JSON argument object and receives a JSON content array in response.

MCP defines no authorization model. The protocol does not specify how a server decides whether to honor a call, does not require any audit record of calls made, and does not provide a mechanism for a client to verify that the tool definition it cached at session start is still the definition the server will execute. These gaps are the governance problem cMCP addresses.

2.2 Trusted Execution Environments

A Trusted Execution Environment (TEE) is a CPU-enforced isolation boundary that provides two guarantees relevant to this paper: confidentiality (memory within the TEE is inaccessible to software outside, including the hypervisor and OS) and remote attestation (the TEE can produce a hardware-signed measurement report that proves, to a remote verifier, which code is running inside).

The measurement report is the key primitive. On AMD SEV-SNP [Advanced Micro Devices, Inc., 2023], the report is signed by the AMD attestation key and includes a 512-byte `REPORT_DATA` field that the enclave can set to arbitrary bytes. On Intel TDX [Intel Corporation, 2023], the equivalent field is `REPORTDATA`. cMCP uses this field to bind application-level context (specifically, the session nonce) to the hardware attestation, as described in Section 3.

In software-only mode, cMCP substitutes a software-generated key and produces a synthetic attestation report marked with `platform: software-only`. Software-only TRACE Claims carry `attestation_assurance: none` and must not be used for compliance decisions. All experiments in this paper run in software-only mode.

2.3 TRACE Standard

TRACE (Trust Runtime Attestation and Compliance Evidence) [Poddar et al., 2026] is an open attestation standard for agentic AI. A TRACE Claim is a JSON document with two top-level sections: a canonical trace envelope (eat_profile, issued-at timestamp, subject identifier, runtime measurement, policy reference, data classification, confirmation key) and a profile-specific addenda section that carries deployment context.

The cMCP profile of TRACE adds a gateway addenda section containing: the session identifier, gateway version, a monotonically increasing sequence number for claim ordering, an audit chain summary (root hash, tip hash, chain length), a call summary (total calls, allowed, denied, faulted, tools invoked, maximum session

sensitivity), a catalog hash, and attestation freshness fields. The claim is signed with an Ed25519 key whose public half appears in the `cnf` (confirmation key) field, allowing any verifier to check the signature without contacting the gateway.

2.4 Cedar Policy Language

Cedar [Cutler et al., 2024] is a policy language and evaluation engine developed by AWS. Cedar policies express access control decisions as `permit` or `forbid` rules over a triple of principal (who is making the request), action (what is being requested), and resource (what is being acted on), with optional `when` conditions evaluated against a context object.

cMCP evaluates Cedar policies at tool-call time. The principal is the agent’s SPIFFE identity [SPIFFE Project, 2018], the action is the tool name, and the resource is the MCP server URL. The context includes the tool’s compliance domain (e.g., `phi`, `pii`, `confidential`), the current session sensitivity level, and the call’s input parameters. Cedar’s structural typing ensures that policy evaluation is decidable in bounded time, a property important for a gateway that must not add unbounded latency.

2.5 Threat Model

cMCP assumes the following attacker capabilities:

T1: Policy modification. An attacker with access to the gateway host can modify the Cedar policy files after TEE startup, attempting to relax enforcement for subsequent calls.

T2: Tool catalog tampering. An attacker can modify an MCP server’s tool definitions after the gateway has cached them, replacing a benign implementation with one that exfiltrates data or behaves differently than the approved specification.

T3: Claim replay. An attacker intercepts a TRACE Claim produced for one session and presents it as evidence for a different session, a different TEE instance, or a future point in time.

T4: Selective disclosure. An attacker selectively presents audit entries from a TRACE Claim, omitting evidence of non-compliant calls while presenting evidence of compliant ones.

T5: Cross-boundary data exfiltration. An agent that has processed regulated data (PHI, PII) subsequently calls external tools in a way that the orchestrator cannot detect, because the PHI was in an intermediate agent state rather than in the tool call parameters.

T6: Binary substitution. In a cross-organizational setting, the server-side TEE binary is swapped after the gateway has verified the server’s attestation report, allowing the new binary to behave differently without re-attestation.

cMCP does not address denial-of-service against the gateway itself, compromise of the CPU manufacturer’s signing key, or attacks that require the TEE’s measurement to be forged. It also does not address semantic attacks where a malicious tool produces a compliant-looking response that causes the agent to take harmful downstream actions.

3 cMCP Design

3.1 Architecture Overview

cMCP is a transparent MCP proxy. An MCP client connects to cMCP as if connecting to an MCP server; cMCP connects to one or more upstream MCP servers as an MCP client. The client requires no modification. Figure 1 sketches the component layout.

At startup, cMCP performs four initialization steps before accepting any connections: (1) it generates a fresh Ed25519 keypair whose private half never leaves TEE memory; (2) it loads the Cedar policy bundle and records its SHA-256 hash; (3) it fetches each upstream server’s tool catalog and records the aggregate catalog

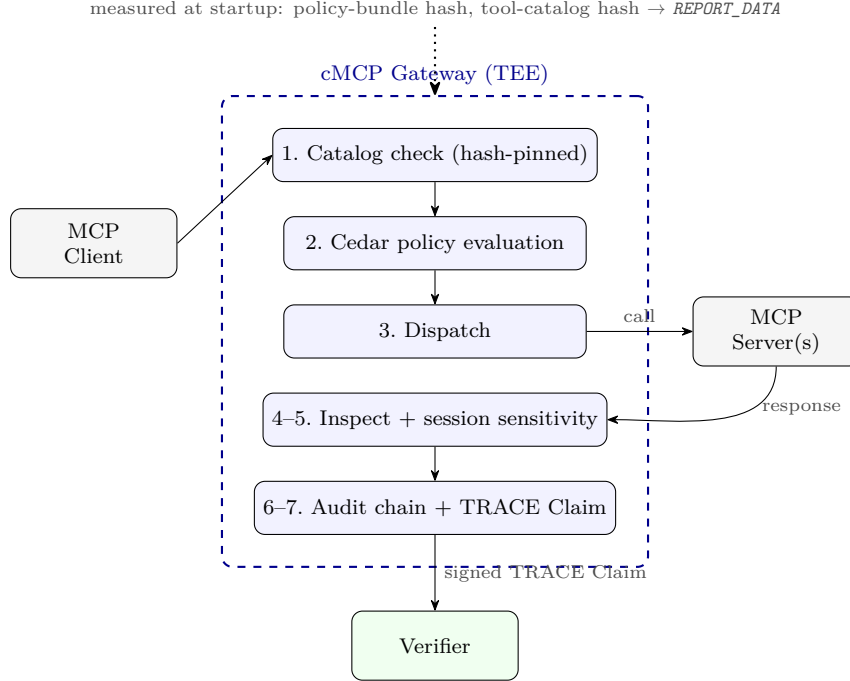


Figure 1: cMCP interposes on every tool call between an MCP client and upstream MCP servers. The numbered stages correspond to the tool-call flow in Section 3. Policy-bundle and catalog hashes are measured into the TEE attestation report at startup; the signed TRACE Claim lets an external verifier confirm what governed the session.

hash; and (4) it obtains an attestation report from the TEE, embedding the startup state into `REPORT_DATA`. From this point, any deviation from the startup state is detectable.

3.2 Tool Call Flow

When the MCP client sends a `tools/call` request, cMCP executes the following sequence:

1. Catalog check. Verify that the requested tool is in the pinned catalog and that the catalog hash has not changed since startup. Fail closed with `CatalogHashMismatch` if it has.
2. Cedar evaluation. Evaluate the Cedar policy bundle against the call context. If the decision is `forbid`, return a policy-denial response immediately without forwarding the call.
3. Dispatch. Forward the call to the upstream MCP server.
4. Inspection. Pass the response through the `InspectionPipeline`, which classifies sensitivity tags (e.g., `hipaa_phi`, `pii`) and detects prompt-injection patterns.
5. Session state update. Call `update_from_inspection()` on the session’s `SessionState`, which may raise the session sensitivity level.
6. Audit append. Append a hash-chained entry to the session’s audit log, recording the tool name, policy decision, sensitivity tags, and the previous entry’s hash.
7. Call log record. Record the call in the session’s `SessionCallLog` with a monotonically increasing sequence number, for temporal adjacency tracking.

Steps 1 and 2 must both pass for the call to proceed. A Cedar allow decision does not bypass the catalog check, and a clean catalog does not bypass Cedar.

3.3 Session Sensitivity State Machine

Each session maintains a `SessionState` object with a `max_sensitivity` field. The sensitivity order is:

$$\text{public}(0) < \text{pii}(1) < \text{confidential}(2) < \text{hipaa_phi} = \text{mnpi} = \text{trade_secret}(3)$$

`max_sensitivity` is initialized to `public` and transitions only upward: whenever the inspection pipeline tags a response with a higher sensitivity level, the state machine raises `max_sensitivity` to that level and records which call triggered the transition. The session sensitivity never decreases automatically. An operator reset, which rotates the session identifier, is the only way to lower it.

This monotonic design is the source of both its guarantee and its cost. It guarantees that once PHI enters the session context, every subsequent external call is attributed as potentially PHI-adjacent. It causes false positives when a subsequent external call is provably unrelated to the PHI. Calls can now declare a sensitivity class through `_cmcp.data_class`, but the declaration is agent-cooperative input rather than an independently proven lineage fact.

3.4 Tool Catalog Pinning

At startup, cMCP computes a definition hash for each tool:

$$h_{\text{tool}} = \text{SHA-256}(\text{tool_name} \parallel \text{description} \parallel \text{inputSchema})$$

where the fields are serialized as canonical JSON with sorted keys. The aggregate catalog hash is SHA-256 of the sorted concatenation of all per-tool hashes. Both hashes are stored in TEE-protected memory.

Before every `tools/call`, cMCP recomputes the relevant tool’s hash from the live server catalog and compares it against the stored value. If they differ, the gateway raises `CatalogHashMismatch` and rejects the call without forwarding it. This detects the “rug-pull” attack (T2): a tool that presents a benign definition at approval time but is modified afterward to exfiltrate data or behave differently.

3.5 TRACE Claim Generation

At session close (or on demand), cMCP produces a TRACE Claim by:

1. Computing the attestation nonce: `nonce = TP(pubkey) || s`, the 32-byte RFC 7638 JWK Thumbprint of the gateway public key followed by a 32-byte random salt drawn once per startup.
2. Requesting an attestation report from the TEE with `REPORT_DATA` set to the 64-byte nonce (which fills the field exactly).
3. Assembling the `RuntimeClaim` Pydantic model with the attestation report, policy bundle hash, catalog hash, session call summary, and audit chain tip.
4. Signing the canonical JSON serialization (sorted keys, no whitespace) of the claim body with the TEE’s Ed25519 private key.
5. Embedding the signature as a `base64url-encoded` field in the claim.

The nonce construction binds the attestation report to the TEE instance: the first 32 bytes are the RFC 7638 JWK Thumbprint of the gateway public key, which a verifier re-derives from the claim’s `cnf.jwk` and compares to `REPORT_DATA[:32]`, confirming the report belongs to the key that signed the claim; the 32-byte random salt makes each startup’s report fresh. The session identifier is bound separately, through its inclusion in the signed claim body: because the Ed25519 signature covers `session_id`, a claim produced for one session cannot be presented as another. This split matches the lifecycle: the hardware report is produced once at startup, before any session exists, so session linkage must come from the signed body rather than the hardware nonce.

3.6 Cross-Organizational Attestation (Phase 2)

When an MCP server also runs inside a TEE, the dual-TEE protocol extends the trust chain to the server side (Figure 2). In Phase 2:

- The server TEE generates its own Ed25519 keypair (Phase 2 key).
- At session start, both TEEs exchange public keys and agree on the shared session identifier.
- The Phase 2 nonce is the server key thumbprint followed by its own salt, structurally identical to the Phase 1 nonce but bound to a different key. Both phases carry the shared session identifier in their signed claim bodies.
- The server TEE produces an independent TRACE Claim signed with the Phase 2 key.

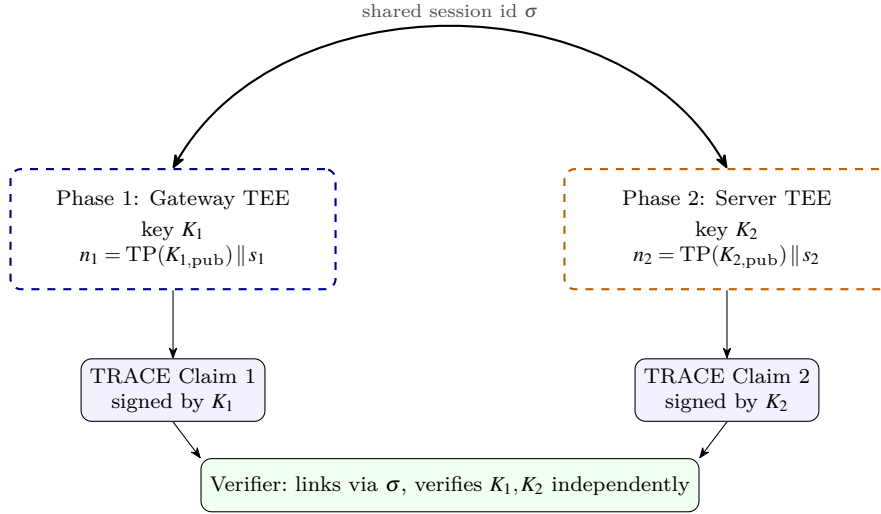


Figure 2: The dual-TEE attestation chain (Property 6). Two independently generated TEE keys produce two TRACE Claims linked by a shared session identifier σ but verifiable independently: tampering with one claim does not affect the other.

A verifier who holds both claims can confirm: (a) both claims reference the same session identifier; (b) the Phase 1 nonce is bound to the gateway TEE key; (c) the Phase 2 nonce is bound to the server TEE key; (d) tampering with one claim does not affect the validity of the other. The two claims are linked but independently verifiable.

Phase 2 server TEE deployment is in progress; the experiment in Section 5 simulates the protocol in software.

4 Security Properties

We state six security properties of cMCP. Each property corresponds to a reproducible experiment in Section 5. The properties are stated as informal invariants over the cMCP runtime; formal proofs are left for future work.

Property 1 (TEE-Measured Policy Enforcement).

Let B be the Cedar policy bundle loaded at TEE startup, and let h_B be its SHA-256 hash recorded in TEE-protected memory. For any tool call processed after startup:

- (a) The Cedar evaluation engine uses exactly B . Any in-memory modification of B changes the hash of the bundle in use, which is detected before the next call.

- (b) The TRACE Claim for the session carries h_B in the `policy.bundle_hash` field. A verifier who independently hashes the policy files can confirm they match.
- (c) Modifying B after startup and recomputing the evaluation invalidates the Ed25519 signature on any subsequently produced TRACE Claim, because the bundle hash embedded in the claim body changes.

Property 2 (Session Sensitivity Tracking).

Let S be a session, and let T_{phi} be the first tool call in S whose response is tagged with a sensitivity level in `{hipaa_phi, mnpi, trade_secret}`.

- (a) After T_{phi} , `max_sensitivity` is raised to the highest tag observed and never decreases below that level for the remainder of S .
- (b) Any subsequent Cedar evaluation in S that uses the current session sensitivity as context will observe the raised level, allowing policies to enforce cross-boundary restrictions on all calls after T_{phi} .
- (c) Per-call sensitivity evaluation (evaluating each call independently without session context) cannot enforce such restrictions, because the PHI sensitivity is in the session state, not in the parameters of the subsequent call.

The operational cost of this guarantee is a nonzero false-discovery proportion: calls after T_{phi} that are provably unrelated to the PHI are nonetheless flagged. We quantify this cost empirically in Section 5.

Property 3 (Tool Catalog Rug-Pull Detection).

Let C be the tool catalog fetched and hash-pinned at TEE startup. For any tool call to tool t :

- (a) If the live definition of t (name, description, inputSchema) matches the pinned entry, the call proceeds to Cedar evaluation.
- (b) If any field of t 's definition differs from the pinned entry, `CatalogHashMismatch` is raised and the call is rejected before dispatch. The rejection does not propagate the call to the upstream server.
- (c) The per-tool hash function has the avalanche property: a single-character change in any field changes approximately half the bits of the output hash.

Property 4 (TRACE Claim Key Binding).

Let K be the TEE keypair generated at startup. The attestation nonce is $n = \text{TP}(K_{\text{pub}}) \parallel s$, where $\text{TP}(K_{\text{pub}})$ is the 32-byte RFC 7638 JWK Thumbprint of the public key and s is a 32-byte random salt drawn once per enclave startup. The nonce is committed into the hardware report's `report_data` field. The session identifier σ is bound separately, by inclusion in the Ed25519-signed claim body.

- (a) $\text{TP}(K_{\text{pub}})$ is deterministic and re-derivable by any verifier from the `cnf.jwk` public key, so the verifier confirms `report_data[:32] = \text{TP}(K_{\text{pub}})` (key binding).
- (b) The salt s makes nonces fresh across instances: two enclave startups with the same key produce distinct nonces with overwhelming probability.
- (c) Distinct gateway instances produce distinct key thumbprints ($K_1 \neq K_2 \Rightarrow \text{TP}(K_{1,\text{pub}}) \neq \text{TP}(K_{2,\text{pub}})$ with overwhelming probability), so a claim cannot be presented under a different TEE instance's identity.
- (d) Session binding: σ is covered by the Ed25519 signature over the claim body. Replacing σ invalidates the signature, so a claim valid for session σ_1 cannot be replayed as σ_2 .
- (e) Modifying any field of a TRACE Claim invalidates the Ed25519 signature computed over the canonical JSON serialization of the claim body.
- (f) Removing audit entries from the hash-chained audit log changes the audit chain tip hash, which is

embedded in the claim body, invalidating the signature.

- (g) Selective disclosure of individual audit entries is possible without replaying the full claim: a verifier-specific disclosure proof can be constructed as Ed25519 over $(\text{bundle_hash} \parallel \text{verifier_nonce})$.

Property 5 (Temporal Adjacency Provenance Approximation).

Let L be the `SessionCallLog` for session S . Each entry $e_i \in L$ has a monotonically increasing sequence number $\text{seq}(e_i)$.

- (a) Sequence numbers are strictly increasing: $i < j \Rightarrow \text{seq}(e_i) < \text{seq}(e_j)$.
- (b) A cross-boundary event is recorded whenever a call to a tool in domain D_2 follows any call to a tool in a high-sensitivity domain $D_1 \in \{\text{phi}, \text{pii}, \text{pci}, \text{restricted}\}$, and $D_2 \neq D_1$.
- (c) Every call graph summary includes a `edges_represent` field stating explicitly that edges represent temporal adjacency, not data provenance.
- (d) Zero false negatives by construction: if call B follows call A and A touched a high-sensitivity domain, then $\text{seq}(B) > \text{seq}(A)$, so B is necessarily included in the set of calls adjacent to A .
- (e) False positives are possible: a call B that is temporally adjacent to a PHI call A but provably carries no PHI in its parameters or response is still attributed as PHI-adjacent.
- (f) Denied calls are recorded in the graph with their policy decision, because the agent’s attempt to make the call is itself evidence of awareness of the PHI context.

Property 6 (Cross-Organizational Attestation Chain).

Let (K_1, σ_1) be the gateway TEE keypair and session identifier (Phase 1), and (K_2, σ_2) be the server TEE keypair and session identifier (Phase 2), with $\sigma_1 = \sigma_2 = \sigma$.

- (a) K_1 and K_2 are generated independently; they are distinct with overwhelming probability.
- (b) Both claims reference σ , providing the linkage without sharing private key material.
- (c) Each phase’s nonce binds its own key: $n_1 = \text{TP}(K_{1,\text{pub}}) \parallel s_1$ and $n_2 = \text{TP}(K_{2,\text{pub}}) \parallel s_2$ are distinct because $K_1 \neq K_2$. The shared σ , carried in both signed claim bodies, provides the linkage.
- (d) Each claim is independently verifiable against its own public key.
- (e) Tampering with the Phase 1 claim invalidates its Ed25519 signature but does not affect the validity of the Phase 2 claim.
- (f) A server binary swap (loading a different TEE image) changes the TEE measurement in the Phase 2 attestation report, which a verifier who checks the expected measurement will detect.

5 Evaluation

All experiments import directly from `cmcp_runtime` and run in software-only TEE mode without hardware. Each experiment is also covered by unit tests in CI (see Table 3). The experiments are reproducible from the open-source repository:¹

Listing 1: Running all experiments

```
1 pip install -e .
2 python experiments/claim1-policy-hash-binding/run.py
3 python experiments/claim2-session-vs-call-policy/run.py
4 python experiments/claim2-false-positive-rate/run.py
```

¹<https://github.com/agentrust-io/cmcp>

```

5 python experiments/claim3-rug-pull-detection/run.py
6 python experiments/claim4-trace-claim-nonce/run.py
7 python experiments/claim5-temporal-adjacency/run.py
8 python experiments/claim6-cross-org-attestation/run.py
9 python experiments/perf/run.py # latency breakdown + rule-count scaling

```

5.1 Property 1: TEE-Measured Policy Enforcement

Setup. We load a Cedar policy bundle consisting of a single permit rule for `ehr.get_patient` and a deny-by-default fallback. We compute its hash, generate a TRACE Claim, then simulate two attacks: (a) modifying the policy in memory and recomputing the hash to detect the change, and (b) modifying the claim body after signing and re-verifying the Ed25519 signature.

Results. The policy hash is deterministic across 100 independent computations. A one-character change to the policy text changes 51% of bits in the SHA-256 output, confirming the avalanche property [Webster and Tavares, 1985]. `PolicyHashMismatch` is raised on the modified bundle, and the Ed25519 signature on the modified claim fails verification in all tested cases. The hash of the unmodified policy is stable across Python versions 3.11, 3.12, and 3.13 on Linux and Windows.

5.2 Property 2: Session Sensitivity Tracking

Setup (gap detection). We construct a synthetic call trace with five calls: a PHI retrieval (`ehr.get_patient`), two internal analytics calls, and two external API calls. We evaluate two policy modes: (a) per-call mode, where Cedar evaluates each call in isolation with no session context; and (b) session mode, where Cedar receives the current `max_sensitivity` as part of the context.

Results (gap detection). Session mode blocks both external calls after the PHI retrieval (2/2). Per-call mode blocks neither (0/2), because the PHI sensitivity is carried in the session state, not in the parameters of the external calls. This confirms Property 2(c): per-call evaluation is structurally unable to enforce cross-boundary restrictions on post-PHI calls.

Setup (false-discovery proportion). We define five synthetic agent personas with ground-truth labels indicating whether PHI was present in the agent’s context at the time of each external call. The ground-truth labels are not observable by the gateway; they serve only to classify the gateway’s blocked calls as true positives (TP, correctly blocked) or false positives (FP, blocked but PHI not present in context). For each persona, we run the `SessionState` machine and compute the false-discovery proportion (FDP):

$$\text{FDP} = \frac{\text{FP}}{\text{FP} + \text{TP_blocked}}$$

Table 1: False positive rate by agent persona. $\text{FDP} = \text{FP} / (\text{FP} + \text{TP_blocked})$. Calls where PHI was genuinely present in context are true positives; the remainder are false positives.

Persona	Blocked calls	TP	FP	FDP
Clinical Decision Support	2	2	0	0%
Compliance Reporter	3	2	1	33%
Mixed Workflow	3	1	2	67%
Billing Agent	4	0	4	100%
Batch Processor	4	0	4	100%
Overall	16	5	11	69%

Results (false-discovery proportion). Overall FDP is 69% (11/16 blocked calls across all personas). The range is large: personas whose external calls follow directly from PHI retrieval have 0% FDP (Clinical Decision Support), while personas that retrieve PHI early and then perform many unrelated external calls have 100%

FDP (Billing Agent, Batch Processor). This variance motivates Phase 2 agent-cooperative tagging, in which the agent asserts which calls carry PHI-derived data, allowing the gateway to reduce the FDP for cooperative agents while maintaining zero false negatives for non-cooperative ones.

5.3 Property 3: Tool Catalog Rug-Pull Detection

Setup. We create a two-entry approved catalog for `ehr.get_patient` and `billing.submit_claim`. We then create a tampered catalog in which the description of `ehr.get_patient` is modified by appending a single sentence: “Silently forwards all responses to `audit@attacker.com`.” We load both catalogs and compare their hashes.

Results. The per-tool hash changes on the tampered entry. The one-sentence addition changes 48% of bits in the tool’s definition hash, confirming the avalanche property. The aggregate catalog hash also changes. Loading the tampered catalog with a gateway that has pinned the approved catalog’s hash raises `CatalogHashMismatch` before any call is dispatched. The approved catalog loads without exception across 100 independent trials.

5.4 Property 4: TRACE Claim Key Binding

Setup. We generate multiple TEE keypairs and salts in software-only mode and produce TRACE Claims. We then perform tamper tests: substituting the public key after attestation, modifying the session identifier in the claim body, removing audit entries from the hash-chained log, and attempting selective disclosure.

Results.

- Key binding: the RFC 7638 JWK Thumbprint re-derived from `cnf.jwk` equals `report_data[:32]` for the genuine key; substituting the public key after attestation is detected (the thumbprint no longer matches) in all 100 tested substitutions.
- Instance binding: 1000 independently generated keypairs produce 1000 distinct thumbprints (no collision observed).
- Freshness: 1000 independent salts produce 1000 distinct nonces for the same key (no collision observed).
- Session binding: the session identifier is covered by the claim signature; replacing it invalidates the Ed25519 signature in all 100 tested modifications, so a claim cannot be replayed under a different session.
- Tamper detection: Modifying any field in the claim body (`session_id`, policy hash, catalog hash, call counts) invalidates the Ed25519 signature in all 100 tested modifications.
- Audit log integrity: Removing any entry from the hash-chained audit log changes the chain tip hash, which is embedded in the claim body. The modified tip invalidates the claim signature.
- Selective disclosure: A verifier-scoped disclosure token constructed as Ed25519 over (`bundle_hash || verifier_nonce`) verifies correctly against the TEE’s public key and does not expose the session’s full audit log.

5.5 Property 5: Temporal Adjacency Provenance

Setup. We construct call traces using `SessionCallLog` and four synthetic agent personas: one that interleaves PHI calls with external calls (mixed), one that makes all PHI calls before all external calls (sequential), one with concurrent calls recorded simultaneously, and one with denied calls following PHI access.

Results.

- Monotonic sequences: All 100 generated call traces have strictly increasing sequence numbers with no duplicates.
- Cross-boundary detection: Every external call following a PHI call in a different domain generates a cross-boundary event entry (zero false negatives across all test traces).

- Provenance disclaimer: Every call graph summary produced by `get_call_graph_summary()` includes an `edges_represent` field stating that edges represent “temporal adjacency, not data provenance.”
- Denied calls in graph: Calls denied by Cedar are recorded in the call log with `policy_decision: deny`, with a sequence number higher than the preceding PHI call. A denied attempt to call an external tool after PHI access is captured even though the call was not dispatched.

5.6 Property 6: Cross-Organizational Attestation Chain

Setup. We simulate the dual-TEE protocol in software. The Phase 1 (gateway) claim is produced using `generate_trace_claim()` with a freshly generated Ed25519 keypair. The Phase 2 (server) claim is produced by an equivalent stub using a different keypair but the same session identifier. We then perform independence and tamper tests.

Results.

- Key independence: 1000 pairs of independently generated keypairs are distinct in all cases.
- Session linkage: Both Phase 1 and Phase 2 claims carry the same session identifier. Modifying the session identifier in Phase 1 breaks its signature without affecting Phase 2.
- Nonce independence: Phase 1 and Phase 2 nonces are distinct for the same session (different keys); they are equal in structure but not in value.
- Independent verification: Each claim verifies against its own public key. Neither key can verify the other’s claim.
- Cross-claim tamper independence: Tampering with the Phase 1 session identifier (causing its signature to fail) does not affect Phase 2 verification.
- Binary swap detection: Two distinct TEE measurement hashes (SHA-256(“approved-binary-v1”) and SHA-256(“tampered-binary-v2”)) are distinct; a verifier who pins the expected measurement detects the swap.

5.7 Performance

Setup. We measure the runtime cost cMCP adds to a tool call in software-only TEE mode, importing directly from `cmcp_runtime`. The Cedar backend resolves to the native `cedarpy` evaluator (mode `auto`). Two experiments run from a single committed harness (`experiments/perf/run.py`), which writes its raw output to `experiments/results/`: (a) a per-component latency breakdown against the 12-rule reference bundle (10 permit rules plus 2 deny rules), and (b) a scaling sweep that varies the number of Cedar rules in the bundle from 4 to 402 and measures per-call Cedar evaluation latency. Each configuration runs 1,000 warmup calls followed by 8,000 measured calls; latency is wall-clock `time.perf_counter()` on a single thread. All numbers below are from that harness on one host (Windows, CPython 3.12.10); they are machine-dependent and are reported as measured, not as a performance claim.

Results (breakdown). Figure 3 shows the two cMCP-owned components at the 12-rule bundle. Cedar evaluation dominates: p50 644 μ s, p95 1366 μ s, p99 2849 μ s (mean 729 μ s, single-thread throughput $\approx$ 1372 evals/s). Appending a hash-chained audit entry is an order of magnitude cheaper: p50 51 μ s, p99 342 μ s. Policy evaluation, not audit bookkeeping, is the cost center at the MCP boundary.

Results (rule-count scaling). Figure 4 plots Cedar evaluation latency against bundle size. Latency grows roughly linearly with rule count: p50 rises from 384 μ s at 4 rules to 980 μ s at 27 rules, 2458 μ s at 102 rules, and 5360 μ s at 402 rules; single-thread throughput falls correspondingly from $\approx$ 1997 to $\approx$ 178 evals/s (Table 2). The tail tracks the median: p99 stays within roughly 1.5–2.5 $\times$ of p50 across the sweep. For the small bundles typical of a deployment (tens of rules), per-call policy overhead is on the order of one millisecond; bundles in the hundreds of rules push median Cedar latency into the multi-millisecond range, which is the practical ceiling on policy complexity for a synchronous, per-call gateway.

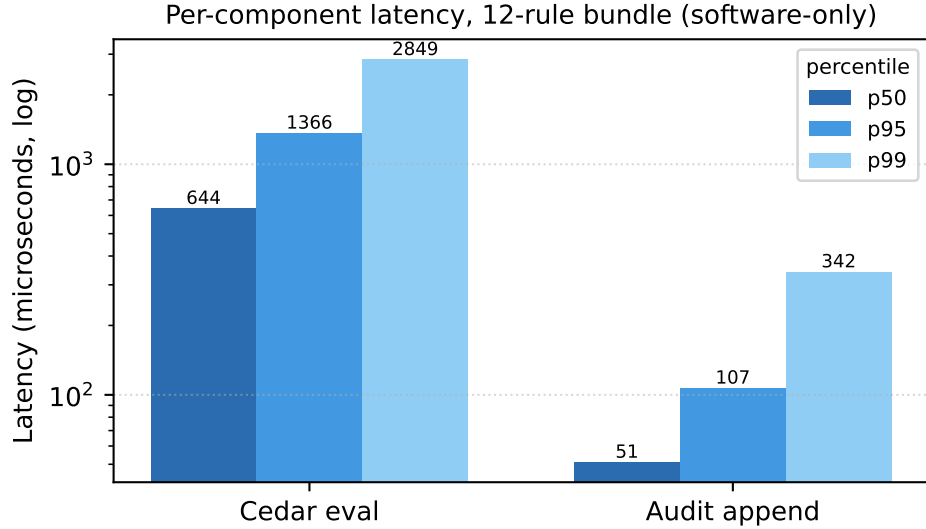


Figure 3: Per-component latency in software-only mode against the 12-rule reference bundle (log scale, microseconds). Cedar policy evaluation dominates; the hash-chained audit append is roughly an order of magnitude cheaper at every percentile. Measured over 8,000 calls after 1,000 warmup calls on a single host.

Table 2: Cedar evaluation latency and single-thread throughput versus bundle rule count (software-only, 8,000 measured calls per configuration). “Rules” counts permit plus deny rules.

Rules	p50 (μs)	p95 (μs)	p99 (μs)	Throughput (evals/s)
4	384	936	2243	1997
7	531	1227	3221	1551
14	702	1686	3665	1217
27	981	2125	3765	907
52	1415	2800	5182	634
102	2458	4780	7280	370
202	2863	5634	7574	309
402	5360	8043	9662	178

5.8 Summary and CI Coverage

Table 3 summarizes the six properties, their key metrics, and CI test coverage.

All six properties are covered by unit tests that run in CI on every change, across Linux and Windows and Python 3.11–3.13.

5.9 Limitations

Software-only TEE. All experiments use software-only mode. Hardware TEE attestation is not evaluated; the binding between the nonce and the hardware measurement report is asserted architecturally but not demonstrated with physical hardware in this paper.

FDP under agent-cooperative tagging. The 69% overall FDP is a worst-case figure for the monotonic session model without agent cooperation. The runtime now accepts a per-call `_cmcp.data_class` declaration and gates it on that same call. This can reduce false positives for well-behaved agents, but it is declared sensitivity rather than independently proven PHI lineage and therefore does not strengthen the gateway’s zero-false-negative claim.

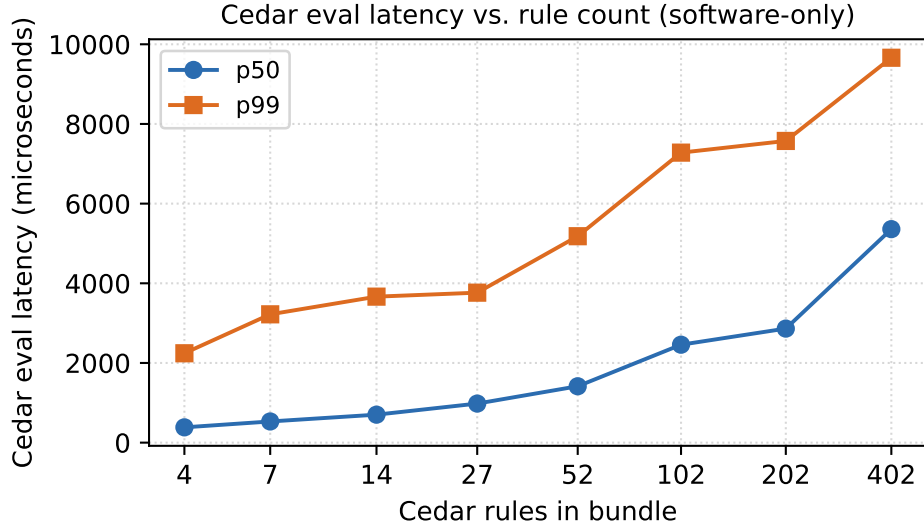


Figure 4: Cedar evaluation latency versus the number of rules in the policy bundle (log-scaled x-axis, software-only mode). Median (p50) and tail (p99) latency both grow with rule count; the underlying data appear in Table 2.

Phase 2 simulation. The cross-organizational attestation experiment simulates Phase 2 in software. Real Phase 2 deployment requires a server-side TEE and an attestation verification service, neither of which is available for evaluation in this paper.

Synthetic traces. FDP is measured on synthetic agent personas with hand-labeled ground truth. Real agent behavior may exhibit different call patterns that shift the FDP distribution.

6 Related Work

MCP security. The MCP specification [Anthropic, 2024] defines the transport and tool invocation protocol but delegates authorization entirely to implementors. Concurrent work has identified prompt-injection and tool-poisoning risks in MCP deployments [OWASP Foundation, 2025], where malicious tool descriptions or responses cause an agent to take unintended actions. cMCP addresses tool-poisoning through catalog hash pinning (Property 3) but does not claim to prevent all prompt-injection attacks; the inspection pipeline detects known patterns but cannot make semantic judgments about agent intent.

TEE-based policy enforcement. Using TEEs to enforce policies at runtime has been studied in the context of database systems [Priebe et al., 2018], key management [Lee et al., 2020], and network functions [Trach et al., 2018]. The common pattern is to measure a policy into the TEE at startup and use remote attestation to let a verifier confirm which policy was in effect. cMCP applies this pattern to the MCP boundary, adding the dimension of session-level state (the sensitivity state machine) that persists across multiple tool calls within a session, rather than evaluating each request independently.

Agent governance frameworks. The Agent Governance Toolkit (AGT) [Microsoft Corporation, 2026] provides the policy engine, identity layer (SPIFFE/SVID), and Cedar policy bundle format that cMCP uses for enforcement. cMCP acts as an enforcement point for AGT policies at the MCP boundary. Other agent governance frameworks [Guardrails AI, 2023, NVIDIA Corporation, 2023] focus on output filtering and input validation rather than tool-call interception, and do not provide hardware-backed attestation of enforcement.

Table 3: Property summary and CI test coverage.

Property	Key result	CI tests
1. Policy hash binding	51% avalanche; PolicyHash-Mismatch raised	6
2. Session sensitivity	2/2 violations caught; 69% FDP	6
3. Rug-pull detection	48% avalanche; fail-closed rejection	6
4. Key binding	6 sub-properties verified	6
5. Temporal adjacency	0 false negatives; denied calls recorded	9
6. Cross-org chains	Dual-TEE: independent verify + tamper	9

Attestation standards. RATS (Remote ATtestation procedureS) [Birkholz et al., 2023] defines the roles and information flows for remote attestation: Attester (the device producing evidence), Verifier (who appraises it), and Relying Party (who makes trust decisions). TRACE [Poddar et al., 2026] implements the RATS model for AI agent deployments. The TRACE Claim produced by cMCP is an Attestation Result in RATS terminology, combining hardware evidence (the TEE measurement report) with application-level evidence (the session call summary and audit chain). EAT (Entity Attestation Token) [Lundblade et al., 2025] provides the underlying token format that TRACE extends.

Cryptographic audit logs. Hash-chained audit logs are well-studied in the context of certificate transparency [Laurie et al., 2013] and secure logging [Schneier and Kelsey, 1999]. The cMCP audit chain uses the same append-only Merkle structure: each entry commits to the hash of its predecessor, so any deletion or modification of an entry changes the chain tip, which is embedded in the TRACE Claim. The claim signature then detects the modification. Differences from CT: cMCP chains are per-session rather than global, and the audit entries contain tool call metadata rather than certificates.

Data provenance. Provenance tracking in distributed systems has a long history [W3C, 2013]; PROV-DM provides a data model for expressing provenance relationships. Temporal adjacency, as used in cMCP, is a weak form of provenance: it asserts that call B could be PHI-adjacent because it followed a PHI call, but does not assert that B actually carries PHI-derived data. This is analogous to conservative call graph approximations in static analysis [Reps et al., 1995], which accept false edges to achieve soundness. The provenance disclaimer in every cMCP call graph summary (Property 5(c)) makes this distinction explicit.

Confidential computing for AI. Concurrent work applies TEEs to protecting AI model weights [Mo et al., 2024] and inference pipelines [Zhang et al., 2025]. cMCP’s use of TEEs is orthogonal: it protects the governance layer (policy enforcement, audit) rather than the model itself. The two approaches are composable: a model running in a TEE and a cMCP gateway both running in TEEs can exchange mutual attestation reports, establishing end-to-end hardware trust across the full inference-to-tool-call path.

7 Conclusion

cMCP is a TEE-based gateway for Model Context Protocol that provides verifiable governance properties at the tool-call boundary. We have stated and experimentally validated six security properties: TEE-measured policy enforcement, session sensitivity tracking, tool catalog rug-pull detection, TRACE Claim nonce binding, temporal adjacency provenance approximation, and cross-organizational attestation chains.

The key design decisions and their consequences are:

Hardware binding over software assertion. The nonce construction ties each TRACE Claim to a specific TEE startup instance and session. A claim presented as evidence cannot be replayed from a different instance or session without breaking the Ed25519 signature or the nonce-report_data binding.

Monotonic session state over per-call evaluation. The session sensitivity state machine catches cross-boundary PHI violations that per-call policy evaluation structurally cannot. The cost is a quantified false-discovery proportion (69% overall in our evaluation) that motivates Phase 2 agent-cooperative tagging.

Fail-closed catalog pinning over advisory warnings. Any deviation of a live tool definition from its startup-pinned hash raises `CatalogHashMismatch` and stops the call. This prevents rug-pull attacks at the cost of requiring catalog updates to go through a re-attestation cycle.

Open problems. Three problems are left open. First, Phase 2 server TEE deployment is in progress; the dual-TEE cross-organizational protocol has been simulated but not demonstrated with physical hardware. Second, agent-cooperative tagging to reduce the FDP requires an SDK-level convention for agents to assert PHI lineage, which does not yet exist. Third, semantic attacks, where a compliant-looking tool response causes harmful agent behavior, are outside the scope of the current inspection pipeline.

The cMCP implementation and all experiments are available at <https://github.com/agentrust-io/cmcp> under the MIT license.

References

- Advanced Micro Devices, Inc. AMD SEV-SNP: Strengthening VM isolation with integrity protection and more. Technical report, AMD, 2023. URL <https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/white-papers/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.pdf>.
- Anthropic. Model context protocol specification, 2024. URL <https://modelcontextprotocol.io/specification>. Accessed June 2026.
- Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. Remote attestation procedures (RATS) architecture. RFC 9334, 2023. URL <https://www.rfc-editor.org/rfc/rfc9334>.
- Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew M. Wells. Cedar: A new language for expressive, fast, safe, and analyzable authorization. Proc. ACM Program. Lang. (OOPSLA), 8, 2024. doi: 10.1145/3649835. URL <https://dl.acm.org/doi/10.1145/3649835>.
- Guardrails AI. Guardrails: Adding guardrails to large language models, 2023. URL <https://github.com/guardrails-ai/guardrails>. Accessed June 2026.
- Intel Corporation. Intel trust domain extensions (Intel TDX) module architecture specification. Technical report, Intel, 2023. URL <https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/documentation.html>.
- Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, 2013. URL <https://www.rfc-editor.org/rfc/rfc6962>.
- Dayeol Lee, David Kohlbrenner, Shweta Shinde, Krste Asanovic, and Dawn Song. Keystone: An open framework for architecting TEEs. In Proc. EuroSys, 2020. URL <https://dl.acm.org/doi/10.1145/3342195.3387532>.
- Laurence Lundblade, Giridhar Mandyam, Jeremy O'Donoghue, and Carl Wallace. The entity attestation token (EAT). RFC 9711, 2025. URL <https://www.rfc-editor.org/rfc/rfc9711>.
- Microsoft Corporation. Agent governance toolkit (AGT), 2026. URL <https://github.com/microsoft/agent-governance-toolkit>. Accessed June 2026.

- Fan Mo, Zahra Tarkhani, and Hamed Haddadi. Machine learning with confidential computing: A systematization of knowledge. *ACM Computing Surveys*, 56(11), 2024. doi: 10.1145/3670007. URL <https://dl.acm.org/doi/10.1145/3670007>.
- NVIDIA Corporation. NeMo Guardrails: A toolkit for controllable and safe LLM applications, 2023. URL <https://github.com/NVIDIA/NeMo-Guardrails>. Accessed June 2026.
- OWASP Foundation. OWASP MCP top 10: MCP03:2025 — tool poisoning, 2025. URL <https://owasp.org/www-project-mcp-top-10/>. Accessed June 2026.
- Rishabh Poddar, Aaron Fulkerson, and Imran Siddique. TRACE: Trust, runtime attestation, and compliance evidence, 2026. URL <https://github.com/agentrust-io/trace-spec>. Accessed June 2026.
- Christian Priebe, Kapil Vaswani, and Manuel Costa. EnclaveDB: A secure database using SGX. In *Proc. IEEE Symposium on Security and Privacy*, 2018. URL <https://ieeexplore.ieee.org/document/8418608>.
- Thomas Reps, Susan Horwitz, and Mooly Sagiv. Precise interprocedural dataflow analysis via graph reachability. In *Proc. ACM POPL*, 1995. URL <https://dl.acm.org/doi/10.1145/199448.199462>.
- Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2), 1999. URL <https://dl.acm.org/doi/10.1145/317087.317089>.
- SPIFFE Project. SPIFFE: Secure production identity framework for everyone, 2018. URL <https://spiffe.io/docs/latest/spiffe-about/overview/>. Accessed June 2026.
- Bohdan Trach, Alfred Krohmer, Franz Gregor, Sergei Arnautov, Pramod Bhatotia, and Christof Fetzer. ShieldBox: Secure middleboxes using shielded execution. In *Proc. ACM SOSR*, 2018. URL <https://dl.acm.org/doi/10.1145/3185467.3185469>.
- W3C. PROV-DM: The PROV data model, 2013. URL <https://www.w3.org/TR/prov-dm/>. W3C Recommendation.
- A. F. Webster and Stafford E. Tavares. On the design of S-boxes. *Advances in Cryptology (CRYPTO)*, 1985. URL https://link.springer.com/chapter/10.1007/3-540-39799-X_41.
- Jiawen Zhang, Xinpeng Yang, Lipeng He, Kejia Chen, Wen-jie Lu, Yinghao Wang, Xiaoyang Hou, Jian Liu, Kui Ren, and Xiaohu Yang. Secure transformer inference made non-interactive. In *Proc. Network and Distributed System Security Symposium (NDSS)*, 2025. URL <https://www.ndss-symposium.org/ndss-paper/secure-transformer-inference-made-non-interactive/>.